Frobenius Lens: Exact Experiment Audits for Periodic Cellular Automata

Shivam Gupta

Independent Researcher, Delhi 110033, India
shivam1720406@gmail.com
ORCID: 0009-0009-4576-0138

Periodic snapshots can conceal differences between local cellular-automaton rules. For scalar linear rules over a prime field, we characterize this ambiguity exactly when observations occur at Frobenius times. Each known-seed experiment induces a polynomial divisibility constraint; their least common multiple determines the complete invisible rule space, observation rank, and compatible-rule count. An impulse probe has an effective ring length controlled by the field characteristic and sampling time. In a binary radius-four example, time-64 snapshots on rings of 32, 64, and 128 cells leave 128 compatible rules; a nine-cell addition distinguishes all 512 rules. Symbolic predictions agree with independent rank calculations on 26,180 exhaustive configurations and 600 mixed suites, with 435 exhaustive fiber checks. A separate browser implementation enables constructive witnesses and experiment repair. The method turns classical cellular-automaton algebra into an exact audit of a declared experiment design.

Key words: cellular automata; finite fields; experimental mathematics; identifiability; periodic simulation; experiment design; reproducible computation.

1 INTRODUCTION

A convincing picture is a useful starting point for mathematical exploration. It is also a measurement produced by a particular initial condition, finite domain, and sampling schedule. Before interpreting agreement between a prototype

and a mathematical model, one should ask what those measurements could distinguish even in principle.

Consider a binary linear cellular automaton with nine possible stencil coefficients, at shifts $-4, \ldots, 4$. An experiment records the full state after 64 updates from a single impulse on each of three periodic rings, of lengths 32, 64, and 128. These are 224 recorded binary cells. Nevertheless, the measurements impose only two independent constraints on the nine coefficients. Every possible output of this experiment has 128 compatible rules. Increasing the last ring to a larger dyadic geometry helps slowly: adding a 256-cell ring leaves 32 compatible rules. Adding an impulse experiment on nine cells instead determines the rule uniquely.

This is not a floating-point error or an unusual random seed. It follows from the interaction of periodic geometry and the characteristic of the field. A laboratory can repeat the same structurally uninformative experiment indefinitely without resolving the ambiguity. An exploration interface should therefore make the ambiguity visible and suggest an experiment that removes it.

We develop *Frobenius Lens*, a workflow and interactive application for this task. The scope is deliberate: one-dimensional, translation-invariant, scalar linear rules over $\mathbb{F}_p$, known initial states, full-ring observations, and observation times p^a, including one. Within this scope the audit is exact and independent of the reference rule. It can be computed before running an experiment.

The contributions are:

1. A joint experiment certificate, expressed as a least common multiple of seed-dependent polynomial constraints, that characterizes every invisible rule difference and every nonempty observation fiber.
2. A constructive repair formulation and exact small-menu search, together with a sharply resolved example in which a nine-cell experiment supplies information missing from much larger rings.
3. A symbolic prime-field implementation, a separate binary browser implementation, an observation consistency and recovery routine, and a standalone simulator used for independent verification.
4. An exhaustive computational study of the certificate on small spaces, mixed-suite checks, an analytic geometry atlas, and an interactive witness demonstration.

The polynomial representation, Frobenius identity, cyclic-ideal algebra, and linear dependence algorithms used here are established mathematics. The contribution is their combination into an explicit audit-and-repair contract for periodic exploration, rather than a claim to discover those underlying mechanisms.

2 RELATED WORK AND THE CONTRIBUTION BOUNDARY

Algebraic cellular automata. Martin, Odlyzko, and Wolfram represent additive periodic automata through Laurent polynomials modulo $x^n - 1$ and analyze periods, predecessors, and transient structure [1]. Their treatment already contains the finite-field and repeated-factor mechanisms responsible for the examples here. Dennunzio et al. study decidability and dynamical properties of substantially broader additive and matrix-valued classes [2]. We use the classical scalar setting to ask a different operational question: which *unknown local rules* are observationally equivalent under a specified collection of experiments?

Rule identification. Zhao and Billings survey identification from spatiotemporal patterns [3]. Sun, Rosin, and Martin estimate local rules and select neighborhoods [4]; Hafiz et al. formulate identification for cellular automata on spaces of Bernoulli probability measures [5]. These works establish identification as a mature problem. Our object is an exact design-side ambiguity description for a restricted linear class, before parameter estimation or neighborhood learning. We assume that the radius bound and coordinate convention are supplied.

Repeated-root algebra. Cyclic codes are described by polynomial ideals, and repeated roots require multiplicity information. Castagnoli et al. develop repeated-root cyclic-code theory and use Hasse derivatives to retain such information [6]. The same warning matters here: replacing a polynomial by its square-free part can discard experiment constraints. Our factor-exponent ceiling is a direct divisibility calculation, not a new factorization or coding-theory algorithm.

Program search and mathematical applications. FunSearch and AlphaEvolve combine program generation with computational evaluators [7, 8]. Recent work also examines evaluation-harness design and exploitation [9]. Our setting isolates one evaluator limitation that exists even with perfectly correct executions: a finite observation map can have a nontrivial kernel. We do not evaluate language-model productivity or the frequency with which coding assistants produce these errors.

Golly, SageMath, and Wolfram Language provide established environments for cellular-automaton exploration [10–12]. Frobenius Lens is a small companion diagnostic: its output is a rule-ambiguity certificate and a repair

TABLE 1
Position of the present work. The table compares questions, not performance.

Line of work	Principal object	Object audited here
Classical CA algebra [1]	Dynamics of a known rule	Ambiguity among bounded unknown rules
Rule identification [3, 4]	Rule/neighborhood estimation	Exact equivalence under a declared design
Repeated-root codes [6]	Code ideals and distance	Frobenius preimages of seed annihilators
Program search [7, 8]	Programs evaluated by a harness	Information retained by a snapshot harness
Exploration environments [10–12]	Simulation and visualization	Invisible changes and additional experiments

suggestion. We make no comparative usability or feature-completeness claim about those systems. Table 1 separates the mathematical objects being studied.

3 RULES, EXPERIMENTS, AND OBSERVATIONS

Let p be prime and $r \geq 0$ a declared radius bound. The rule class is

$$\mathcal{F}_r = \left\{ f(x) = \sum_{j=-r}^{r} c_j x^j : c_j \in \mathbb{F}_p \right\}. \tag{1}$$

The bound is inclusive: endpoint coefficients may vanish. Thus $|\mathcal{F}_r| = p^{2r+1}$. Multiplication by f means the local update

$$u_i^{(k+1)} = \sum_{j=-r}^{r} c_j u_{i-j}^{(k)} \pmod{p}, \qquad i \in \mathbb{Z}/n\mathbb{Z}. \tag{2}$$

The sign of the shift is fixed by this equation. We do not identify rules up to reflection, spatial shift, or a change of origin.

An experiment $e = (n, t, s)$ consists of a positive ring length n, a time $t = p^a$ with $a \geq 0$, and a known initial-state polynomial $s \in \mathbb{F}_p[x]$ of degree less than n. The zero seed is permitted. Write $M_n = x^n - 1$ and $R_n = \mathbb{F}_p[x]/(M_n)$.

Since x is a unit in R_n, Laurent rules act on this quotient without ambiguity. The full observation is

$$O_e(f) = f(x)^t s(x) \pmod{M_n}. \tag{3}$$

For a finite suite E, O_E concatenates the full observations from its experiments. Each experiment starts afresh from its specified seed; snapshots are indexed by absolute time from that seed. The observations are exact field elements, rather than rendered pixels, histograms, or noisy measurements.

Define the normalized rule $F = x^r f \in \mathbb{F}_p[x]$, with $\deg F \le d := 2r$. For two rules f, g, write $H = x^r(f - g)$. At a Frobenius time,

$$f^t - g^t = (f - g)^t = x^{-rt} H^t. \tag{4}$$

Consequently, O_E is linear over $\mathbb{F}_p$ in the $d + 1$ rule coefficients. This is linearity in the *rule*, in addition to the usual linearity in the state. Equation (4) is the reason for restricting the time domain. It generally fails at other times.

4 AN EXACT CERTIFICATE FOR THE WHOLE SUITE

For each experiment define

$$q_e = \frac{M_n}{\gcd(M_n, s)} = \prod_{\phi} \phi^{\nu_{e,\phi}}, \qquad Q_e = \prod_{\phi} \phi^{\lceil \nu_{e,\phi}/t \rceil}, \tag{5}$$

where the factors are monic irreducible polynomials over $\mathbb{F}_p$. Empty products are one. Define the joint polynomial

$$L_E = \operatorname{lcm}_{e \in E} Q_e, \qquad D_E = \deg L_E, \tag{6}$$

with $L_\varnothing = 1$. This certificate depends only on the design, not on the unknown rule or its measured output.

Theorem 1 (Complete ambiguity certificate)**.** *For every suite E in the stated scope and every $f, g \in \mathcal{F}_r$,*

$$O_E(f) = O_E(g) \iff L_E \mid x^r(f - g). \tag{7}$$

The coefficient-space kernel has dimension

$$\nu_E = \max\{0, d + 1 - D_E\}, \qquad \operatorname{rank} O_E = d + 1 - \nu_E. \tag{8}$$

Every nonempty observation fiber contains exactly p^{ν_E} rules. The suite distinguishes every rule in $\mathcal{F}_r$ if and only if $D_E > d$. When $\nu_E > 0$, a basis of normalized invisible differences is

$$L_E, xL_E, \ldots, x^{d - D_E} L_E. \tag{9}$$

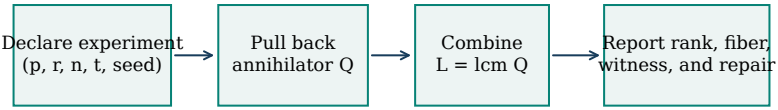


FIGURE 1
The audit contract. A design determines Q_e and L_E before any unknown-rule observations are used. Independent numerical paths verify the bounded computational study.

Proof. Fix an experiment and put $M = M_n$. Equations (3)–(4) show that equality of observations is equivalent to $M \mid H^t s$, since x^{-rt} is a unit modulo M.

Let $z = \gcd(M, s)$, $M = zq$, and $s = zs'$. Then $\gcd(q, s') = 1$, so $M \mid H^t s$ is equivalent to $q \mid H^t$. For every irreducible ϕ, this is equivalent to $t\, v_\phi(H) \geq v_\phi(q)$, or $v_\phi(H) \geq \lceil v_\phi(q)/t \rceil$. Hence the equality holds exactly when $Q_e \mid H$. Requiring this for all experiments gives $L_E \mid H$.

If $D_E > d$, no nonzero polynomial of degree at most d is divisible by L_E. Otherwise the admissible multiples are exactly $L_E B$, with $\deg B \leq d - D_E$. Multiplication by the nonzero polynomial L_E is injective in $\mathbb{F}_p[x]$, and the monomials in B give the basis (9). This establishes the kernel dimension and, by rank–nullity, the rank. A nonempty fiber of a linear map is a coset of its kernel, so its size is p^{ν_E}. □

Theorem 1 characterizes a finite model class exactly. It does not infer that a black-box program implements a member of that class. If observations are inconsistent with the class, there is no compatible rule and the fiber-size statement does not apply. The software checks consistency separately when recovering coefficients.

4.1 Impulse probes and the effective ring length

Corollary 2 (Exact impulse reduction)**.** *For an impulse $s = 1$, let $n = p^b m$ with $p \nmid m$ and $t = p^a$. Then*

$$Q_e = (x^m - 1)^{p^{\max(b-a,0)}} = x^{n/p^{\min(a,b)}} - 1. \tag{10}$$

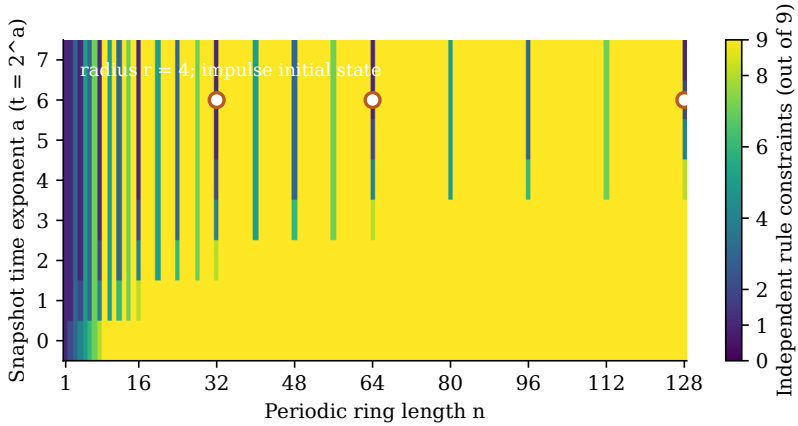


FIGURE 2
Exact analytic rank for a binary radius-four rule under one impulse experiment. The color is $\min(9, n/2^{\min(a,v_2(n))})$. Circled cells mark the three time-64 experiments in the case study. The image shows a formula, not a sampled error rate.

An impulse suite therefore uses the least common multiple of the polynomials in (10)*. A single impulse experiment is identifying exactly when* $n/p^{\min(a,b)} > 2r$.

Proof. In characteristic p, $x^n - 1 = (x^m - 1)^{p^b}$. Since $p \nmid m$, the polynomial $x^m - 1$ is square-free: its derivative mx^{m-1} is relatively prime to it. Every irreducible multiplicity is p^b. Equation (5) replaces it by $\lceil p^b/p^a \rceil = p^{\max(b-a,0)}$. Applying Frobenius again gives the final expression. Theorem 1 supplies the threshold. □

For $p = 2$, a ring of 2^b cells observed at 2^a with $a \geq b$ supplies only the constraint $x - 1$. In contrast, an odd ring retains $Q_e = x^n - 1$ at every supported time. The geometry atlas in Figure 2 shows this arithmetic structure. Neighboring ring sizes can have very different information content even when they look equally adequate as display resolutions.

4.2 What additional seeds and later snapshots can change

Proposition 3 (Limits of repetition)**.** *On a fixed ring, the union of any seed experiments at a fixed Frobenius time cannot impose a stronger constraint than an impulse at that time. For a fixed seed and ring, adding later Frobenius*

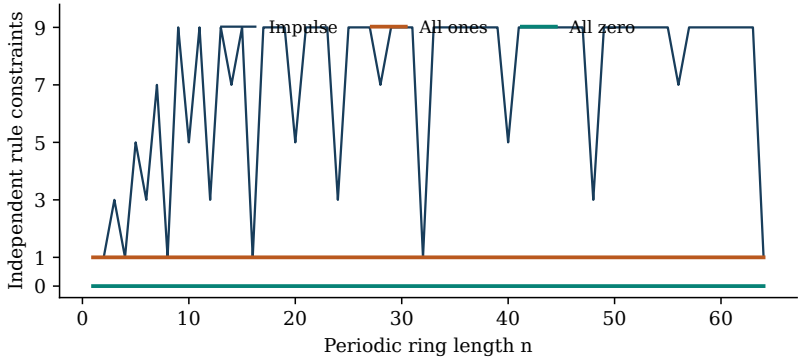


FIGURE 3
Seed effects for binary radius four at time 64. Full-ring observations of all-one states constrain only the coefficient sum, even on odd rings. All-zero states impose no constraint. Impulse probes expose the geometry-dependent maximum for a fixed ring and time.

times to an earlier Frobenius snapshot adds no rule information. An all-one initial state gives $Q_e = x-1$ *at every supported time; a zero state gives* $Q_e = 1$.

Proof. For every seed, $q_e \mid M_n$, so the factor-exponent ceilings in its Q_e do not exceed those of the impulse constraint. Their least common multiple also divides that constraint. At a later Frobenius time, each ceiling is no larger than at the earlier time, so the later constraint divides the earlier one. Finally, the all-one seed is $s = (x^n - 1)/(x - 1)$, giving $q_e = x - 1$, whose ceiling remains one. A zero seed has $q_e = 1$. □

This does not say that extra seeds are useless when the original seed is weak. An impulse can recover information omitted by a constant state. It says that once an impulse has supplied the entire operator response at a fixed time, more initial states cannot recover a rule difference whose time-t operator is already zero. Figure 3 distinguishes these two sources of lost information.

Proposition 4 (Detection by a random seed)**.** *Fix distinct rules with normalized difference* H*, and a ring/time pair* (n, t) *in scope. If the initial state is uniform on* $\mathbb{F}_p^n$*, the probability of equal observations is*

$$\Pr\{O_e(f) = O_e(g)\} = p^{-\left(n - \deg \gcd(M_n, H^t)\right)}. \tag{11}$$

For k independently chosen initial states on the same ring and time, the exponent is multiplied by k. If $M_n \mid H^t$, the probability remains one for every k.

Proof. Put $G = \gcd(M_n, H^t)$. The condition $M_n \mid H^t s$ is equivalent to $(M_n/G) \mid s$. Exactly $p^{\deg G}$ polynomials of degree less than n satisfy this condition. Divide by p^n and use independence for repetitions. □

This is a probability conditional on an explicitly uniform seed model. It is not an estimated prevalence of errors in real mathematical software.

5 REPAIRING AN EXPERIMENT DESIGN

Let C be a finite menu of additional experiments with positive costs w_e. Given an existing suite E, a complete repair solves

$$\min_{S \subseteq C} \sum_{e \in S} w_e \quad \text{subject to} \quad \deg \operatorname{lcm}\big(L_E, \{Q_e : e \in S\}\big) > d. \tag{12}$$

The cost can represent stored cells, simulation work, or an externally supplied resource measure. These are different objectives. The case study uses $w_e = n_e$, the number of newly stored snapshot cells. It does not equate this quantity with runtime.

A simple necessary lower bound is

$$\sum_{e \in S} n_e \geq \max(0, d + 1 - D_E), \tag{13}$$

because $\deg \operatorname{lcm}(L_E, \{Q_e\}) \leq D_E + \sum_e \deg Q_e \leq D_E + \sum_e n_e$. Overlap between factors can make the bound strict. In particular, summing individual experiment degrees without subtracting common factors is not a valid certificate.

The reference implementation enumerates subsets and retains the cheapest feasible repair; it caps the menu at 22 entries. This is an exponential exact search for small menus. We do not claim a polynomial-time global design algorithm. The browser demonstration instead scans increasing ring lengths for the cheapest *single impulse* addition in a finite range. A result from that scan is not labeled optimal over all seeds, times, or multisets of experiments.

5.1 A fully resolved radius-four example

Set $p = 2$, $r = 4$, and take impulse experiments on rings 32, 64, and 128, all at time 64. Their constraints are $x + 1$, $x + 1$, and $(x + 1)^2$, so $L_E = x^2 + 1$,

$D_E = 2$, and $\nu_E = 7$. A concrete invisible normalized difference is $H = x^2 + 1$. For example,

$$F = 1 + x^2 + x^3 + x^6 + x^8, \tag{14}$$

$$G = F + H = x^3 + x^6 + x^8. \tag{15}$$

The corresponding physical rules are $f = x^{-4}F$ and $g = x^{-4}G$. Their histories can differ at intermediate times while their declared snapshots agree (Figure 4).

For an additional nine-cell impulse experiment at time 64, $Q = x^9 + 1$ and

$$\operatorname{lcm}(x^2 + 1, x^9 + 1) = (x + 1)(x^9 + 1) = x^{10} + x^9 + x + 1. \tag{16}$$

Its degree ten exceeds eight; therefore all 512 rules are distinguished. The fixed witness differs in two of the nine final cells.

Among additional single impulse rings at this same time, nine is the smallest successful length. For $n \le 8$, the effective lengths are $1, 1, 3, 1, 5, 3, 7, 1$. Combining each resulting constraint with $x^2 + 1$ gives degree at most eight. Length nine gives degree ten. If every new ring must have at most seven cells, rings three and seven together achieve degree ten and cost ten. Exhaustive subset search of the seven-entry menu proves that no lower-cost subset in that menu is sufficient. These are concrete optimality statements under declared menus and costs.

6 IMPLEMENTATION AND INDEPENDENT VERIFICATION

The Python implementation uses exact polynomial gcd, factorization, and least common multiple operations in SymPy 1.14.0. For each experiment it constructs q_e, replaces every irreducible multiplicity ν by $\lceil \nu/t \rceil$, and accumulates L_E. It exports field coefficients in increasing degree order, experiment specifications, the joint generator, rank, nullity, fiber size, and a kernel basis. The core certificate procedure is:

A separate `recover` routine forms the linear observation equations, rejects inconsistent full-ring observations, and returns one compatible coefficient vector plus the ambiguity certificate. Recovery uses modular Gaussian elimination. It does not force a unique answer when the design is underdetermined.

The browser implementation is restricted to $p = 2$ and uses BigInt polynomial arithmetic. It obtains Q_e without factorization: starting with $s, x^t s, x^{2t} s, \dots$ modulo M_n, it finds the first vector dependence, tracking the

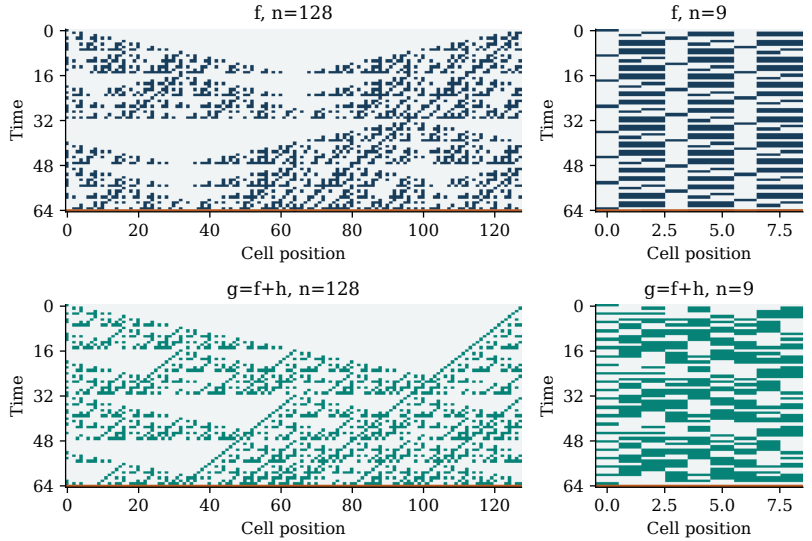


FIGURE 4
Direct simulation of the two displayed rules from an impulse. The horizontal line marks time 64. On the 128-cell ring, both final states coincide despite different histories. On nine cells, their final states differ. These images visualize an exact witness established algebraically and checked by repeated local updates.

dependence coefficients. This is a standard Krylov dependence construction. At binary Frobenius times the dependence polynomial is precisely the smallest-degree monic H with $H^t s = 0$, hence Q_e. Section A gives the argument. Euclidean polynomial operations then produce the joint least common multiple.

The independent Python verifier imports neither SymPy nor the certificate module. It implements Equation (2) with lists, constructs observation-matrix columns by evolving each coefficient-basis rule, and calculates rank by modular row reduction. Exhaustive fiber checks instead enumerate every rule and group identical simulated outputs. These paths have different failure modes: symbolic factor arithmetic, local stepping, linear elimination, and output partitioning.

The artifact includes a protocol file, raw per-configuration results, deterministic random seeds, code, figure-generation scripts, and test summaries. It is

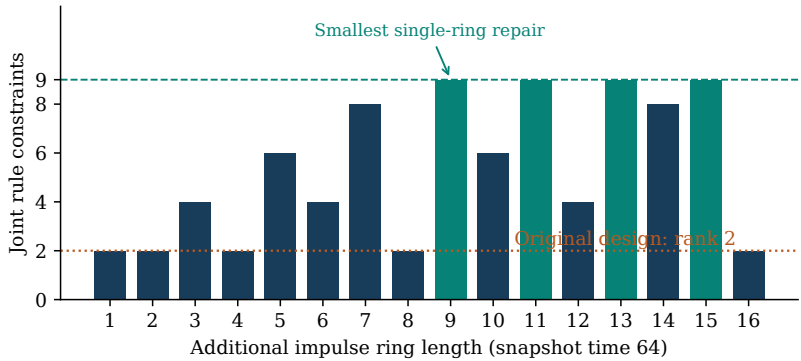


FIGURE 5
Joint rank after adding one impulse experiment at time 64 to the original three-ring suite. Nine is the smallest additional ring attaining rank nine. More recorded cells do not imply a more informative geometry: powers of two remain weak at this time.

designed for replay rather than reliance on screenshots or aggregate numbers alone.

7 COMPUTATIONAL RESULTS

All reported experiments were executed with Python 3.12.14 on an arm64 macOS machine. The primary run took 69.14 seconds. This duration describes one local run and is not a portable speed guarantee. The protocol hash and raw results are in the repository.

7.1 Exhaustive small-space audit

The primary grid covers every binary seed for ring lengths $1 \le n \le 9$, times $1, 2, 4, 8$, and radii $0, 1, 2, 3$: 16,352 configurations. A second grid covers every ternary seed for $1 \le n \le 6$, times $1, 3, 9$, and radii $0, 1, 2$: 9,828 configurations. Zero seeds and rings shorter than the stencil are included. Predicted rank agrees exactly with independently simulated rank in all 26,180 configurations.

For mixed designs, 400 binary and 200 ternary suites use a deterministic pseudorandom generator seeded with 1720406. Each contains one to four experiments; radii range from zero through eight, ring lengths from one through

Listing 1
Certificate construction; all polynomial operations are over the declared prime field.

```
L = 1
for (n, t, seed) in experiments:
    require t is a power of p
    M = x**n - 1
    q = exact_quotient(M, gcd(M, seed))
    Q = product(phi**ceil(v/t) for phi, v in factor(q))
    L = monic(lcm(L, Q))
nullity = max(0, 2*radius + 1 - degree(L))
return L, nullity, [x**j * L for j in range(nullity)]
```

24, and time exponents from zero through three. There are no rank disagreements. Random here describes the constructed design generator, not an assumption used in Theorem 1.

An additional 435 configurations enumerate the entire rule class and compare actual fiber sizes to p^{ν_E}. Binary radii one through three use rings one through six, four supported times, and five seed choices. Ternary radius one uses rings one through five, three times, and five seed choices. The choices are zero, impulse, constant, alternating, and fixed pseudorandom; duplicate seeds are retained and counted as repeated test configurations. Every partition has the predicted number and uniform size of nonempty fibers.

The 2,304-point impulse grid compares the symbolic certificate with Equation (10) for $p \in \{2, 3, 5\}$, $1 \le n \le 128$, and $0 \le a \le 5$. It is a cross-check of two algebraic formulations, whereas the exhaustive small grids use independent local simulation. Table 2 preserves that distinction. Ten additional unit-test methods cover boundaries, duplicates, empty suites, inconsistent data, unsupported domains, and recovery. The recovery method tests 180 field/radius/ring/time combinations.

The browser kernel agrees with all 16,352 binary rank/degree fixtures. It also agrees with repeated JavaScript time stepping on 1,536 rule/geometry/-time combinations and passes two case-study fiber checks, for 17,890 checks in total. UI inspection confirms that applying the suggested nine-cell repair changes the displayed rank from two to nine and the fiber size from 128 to one.

TABLE 2
Executed exact checks. Counts refer to assertions or design configurations as specified, not independent statistical samples.

Check	Count	Failures
Exhaustive single-experiment rank configurations	26,180	0
Mixed-suite rank configurations	600	0
Polynomial snapshot vs. repeated local updates	1,545	0
Constructive kernel witnesses	975	0
Exhaustive observation-fiber configurations	435	0
Impulse formula grid ($p = 2, 3, 5$)	2,304	0
Case-study fibers and repair searches	7	0
Input guards and non-Frobenius counterexample	5	0
Primary Python assertions	32,051	0
Separate browser-kernel checks	17,890	0

7.2 Exhaustive repair comparison

Table 3 enumerates every one of the 512 radius-four binary rules for each design, relative to the fixed target in Section 5.1. An alternative is missed exactly when its entire recorded output equals that of the target. These are exhaustive counts over the declared class; they are not measurements of naturally occurring implementation errors.

The earlier snapshot is a valid and effective control. The repair's advantage in this example is that it preserves the late observation time while using fewer additional snapshot cells. Recording the first update is often the simpler choice when the interface permits it. The result does not justify preferring late sampling in general.

7.3 Descriptive computation cost

For binary radius four at time 64, five local repeats gave median cached-certificate times of 0.92, 1.02, 0.92, and 0.87 milliseconds for ring lengths 32, 64, 128, and 256. Independent basis simulation and rank computation took 4.90, 10.64, 19.32, and 30.54 milliseconds. The certificate timings reuse the cached per-experiment factorization; the basis simulation timings perform time stepping. They are different tasks, not an end-to-end algorithmic speed comparison. The raw repeats are provided to make this qualification inspectable. The mathematical information result does not depend on these timings.

TABLE 3
Exact design comparison. Added cells count newly stored full snapshots, beyond the original 224 cells. Each row starts from the same original suite.

Additional experiment(s)	Added cells	Rank	Fiber	Missed alternatives
None	0	2	128	127
$n = 256$, $t = 64$, impulse	256	4	32	31
$n = 128$, $t = 1$, impulse	128	9	1	0
$n = 9$, $t = 64$, impulse	9	9	1	0
$n = 3, 7$, $t = 64$, impulses	10	9	1	0

8 INTERACTIVE USE AND PRACTICAL APPLICATIONS

A workshop demonstration begins with the three-ring design and asks whether 224 recorded cells determine nine coefficients. The app reports two independent constraints, an ambiguity dimension of seven, and 128 compatible rules. It displays $L_E = x^2 + 1$ and a concrete pair of distinct rules with identical recorded snapshots. A user can inspect their histories, request a repair, and apply the nine-cell experiment. The witness is retained after the repair, making the newly detected difference visible.

Changing the initial state to all ones exposes a different issue: a geometrically adequate ring can still provide only one constraint. Changing the radius shows the boundary between a complete and an incomplete design. Exported JSON records the experiment assumptions, coefficients of L_E, rank, nullity, and fiber size. The local browser app requires neither a server-side model nor a network connection after loading its two static files.

There are three concrete integration opportunities. First, a mathematical notebook can attach the certificate to a conjecture-generating simulation so that the domain and sampling assumptions remain visible. Second, a regression harness for finite-field stencil software can choose a complete impulse design rather than accumulate redundant late snapshots. Third, an educational environment can turn an apparently anomalous simulation into a constructive lesson about characteristic, repeated roots, and identifiability.

A commercial implementation could package these checks as a notebook extension or testing component for computational-science tooling. The present artifact establishes an exact restricted capability and a reproducible demonstration. It does not establish customer demand, willingness to pay,

a market-size estimate, or superiority over existing commercial tools. The adjacent product landscape contains mature simulators and symbolic environments [10–12]; a credible product hypothesis is integration with these workflows. A next validation step would be a corpus of real simulation revisions with separately verified rule-class membership and resource accounting.

9 LIMITATIONS AND SCOPE OF THE EVIDENCE

The strongest conclusion is conditional and finite: under the declared scalar linear class and exact full observations, the certificate completely characterizes rule ambiguity. Several boundaries matter in use.

Rule-class membership. A nonlinear implementation, an asynchronous update, a boundary bug, or a state-dependent stencil can agree on the certified experiments without belonging to $\mathcal{F}_r$. One must establish the intended class separately, for example through source inspection or a restricted implementation language. A complete certificate for $\mathcal{F}_r$ is not a proof of arbitrary-program correctness.

Observation scope. Cropped views, screenshots, population counts, noisy measurements, unknown initial states, and unknown spatial alignment are outside the theorem. The software consumes full vectors of exact field elements. A rendered figure is an illustration, not the observation oracle.

Time and algebra. The proof uses $t = p^a$. At time three in characteristic two, $(f + g)^3$ generally differs from $f^3 + g^3$; the artifact includes a concrete counterexample and rejects that time in the certificate API. Matrix-valued rules introduce noncommuting coefficient operations, and multidimensional polynomial rings do not generally admit a single principal generator. Neither extension follows from the present proof.

Novelty and scale. The algebra is classical. The proposed contribution is an executable, seed-aware, multi-experiment audit with complete ambiguity reporting and repair. The literature review supports this positioning; it cannot prove that every equivalent formulation is absent from the literature. The exhaustive checks are finite and share ordinary software/hardware risks. They corroborate the proofs but do not replace them, nor do they establish widespread practical error rates.

Design optimization. The subset search is exponential and intended for small menus. Snapshot-cell cost omits many real deployment costs. A more expensive experiment in this metric can be preferable when runtime, visualization, or existing infrastructure dominates.

10 CONCLUSION

Periodic experiments should be accompanied by a statement of what they distinguish. For scalar linear cellular automata at Frobenius times, that statement can be exact: a polynomial L_E determines every invisible rule change, the observation rank, and the size of every compatible-rule set. The nine-cell repair demonstrates why a small change in experiment geometry can resolve ambiguity that persists across much larger simulations. Frobenius Lens makes this algebra available as a working exploration tool, with proofs, raw finite checks, and separate implementations that can be inspected and reproduced.

Code and data availability. The repository at `https://github.com/shi1720/frobenius-lens` contains source, protocol, raw results, tests, browser app, and figure/manuscript sources. Software is released under the MIT license. Generated results are deterministic except for recorded wall-clock timings. No proprietary data are required.

A FACTORIZATION-FREE BINARY CERTIFICATE

For a binary experiment (n, t, s) with $t = 2^a$, let $V_j = x^{jt}s \pmod{x^n - 1}$. Perform exact elimination on $V_0, V_1, \ldots$, retaining the binary linear combination used in every elimination. At the first dependence index k, write

$$V_k + \sum_{j=0}^{k-1} b_j V_j = 0, \qquad B(x) = x^k + \sum_{j=0}^{k-1} b_j x^j. \tag{17}$$

Frobenius gives $B(x)^t s = V_k + \sum_j b_j V_j = 0$. Therefore $Q_e \mid B$ by the argument of Theorem 1. Conversely, $Q_e^t s = 0$ supplies a dependence at index $\deg Q_e$. Minimality implies $k = \deg Q_e$, and the two monic polynomials agree. A dependence exists by index n, since all vectors lie in an n-dimensional space. For a zero seed it occurs at index zero and returns $Q_e = 1$.

The browser represents vectors and dependence polynomials as BigInts, uses bitwise xor for addition, and implements binary Euclidean division for gcd and least common multiple. The algorithm is deterministic. Its worst-case work grows with the ring dimension and is not claimed to improve on specialized algebraic algorithms. The UI caps ring length at 512 and the displayed radius at 12; the kernel API accepts radii up to 32.

B A NON-FROBENIUS COUNTEREXAMPLE

Take $p = 2$, $n = 7$, and radius one, with normalized coefficient vectors $(1, 1, 0)$ and $(0, 1, 1)$. In Laurent notation these are $f = x^{-1} + 1$ and $g = 1 + x$. At time three,

$$(f + g)^3 - f^3 - g^3 = f^2 g + f g^2 = x^{-2} + x^2 \neq 0 \pmod{x^7 - 1}. \tag{18}$$

An impulse therefore observes a nonlinear-in-rule map. The linear fiber argument must not be extended to this time by simply replacing t in Equation (5). For example, the factorization $f^t - g^t = (f - g)^t$ used in the main proof is already false.

C REPRODUCTION MAP

The artifact paths are relative to the repository root.

Path	Purpose
`src/frobenius_lens.py`	Exact prime-field certificate, snapshots, repair, recovery
`tests/independent.py`	Separate local-update simulator and modular rank
`experiments/protocol.json`	Primary finite grids, seed, and case specification
`experiments/run.py`	Execute primary checks and save raw results
`results/single_checks.csv`	Every exhaustive single-design rank comparison
`results/suite_checks.json`	Mixed-suite specifications and certificates
`results/fiber_checks.json`	Exhaustive output-partition results
`results/case_study.json`	Compatible rule IDs and exact repair outcomes
`app/lens.mjs`	Factorization-free binary browser kernel
`tests/test_browser_kernel.mjs`	Browser-kernel fixture and time-stepping checks
`experiments/figures.py`	All five vector figures in the manuscript

A minimal exact audit is:

```
from frobenius_lens import Experiment, certificate
E = [Experiment(n, 64) for n in (32, 64, 128)]
assert certificate(E, radius=4)["fiber_size"] == 128
E.append(Experiment(9, 64))
assert certificate(E, radius=4)["fiber_size"] == 1
```

The primary protocol SHA-256 is

`28873bd3abc7d1cf75668cd251f76595b6d`
`1642ff49db10c143cc55ba60495c1.`

This identifies the protocol text, not a public preregistration. The UI and recovery checks were added as implementation validation alongside the primary study.

NOMENCLATURE

Symbol	Meaning
$p, \mathbb{F}_p$	Prime characteristic and its field.
$r, d = 2r$	Radius bound and normalized rule-degree bound.
$f, g, \mathcal{F}_r$	Laurent rules and the bounded rule class.
$c_j, u_i^{(k)}$	Stencil coefficient and cell state after k updates.
$n, t = p^a$	Ring length and Frobenius observation time.
$s, e = (n, t, s)$	Known initial state and one experiment.
E, O_E	Experiment suite and its concatenated observation map.
M_n, R_n	$x^n - 1$ and the quotient $\mathbb{F}_p[x]/(M_n)$.
F, G, H	Normalized rules $x^r f, x^r g$ and difference $F - G$.
ϕ, v_ϕ	Monic irreducible factor and its multiplicity valuation.
q_e, Q_e	Seed annihilator and its Frobenius pullback.
L_E, D_E	Joint least common multiple and its degree.
ν_E	Dimension of the coefficient-space kernel.
$b = v_p(n), m$	Characteristic valuation of n and n/p^b.
C, S, w_e	Candidate menu, selected repair, and positive cost.
V_j, B	Krylov vector and its first dependence polynomial.

All algebraic quantities are dimensionless. Observation time is counted in discrete updates. Reported execution times use seconds or milliseconds.

AVAILABILITY AND COMPETING INTERESTS

The code, raw results, protocol, and mathematical figures are available at https://github.com/shi1720/frobenius-lens; the reproducible software release is v0.1.0. The software license is MIT. The author is founder of Siloed, an AI software consultancy; the manuscript discusses possible applications in mathematical-software tooling. No external research grant is reported for this study.

REFERENCES

[1] Olivier Martin, Andrew M. Odlyzko, and Stephen Wolfram. (1984). Algebraic properties of cellular automata. *Comm. Math. Phys.*, 93(2):219–258. https://doi.org/10.1007/BF01223745.

[2] Alberto Dennunzio, Enrico Formenti, Darij Grinberg, and Luciano Margara. (2020). From linear to additive cellular automata. In *47th International Colloquium on Automata, Languages, and Programming (ICALP 2020)*, volume 168 of *Leibniz International Proceedings in Informatics*, pages 125:1–125:13. https://doi.org/10.4230/LIPIcs.ICALP.2020.125.

[3] Y. Zhao and S. A. Billings. (2006). The identification of cellular automata. Technical Report 938, University of Sheffield, Department of Automatic Control and Systems Engineering.

[4] Xianfang Sun, Paul L. Rosin, and Ralph R. Martin. (2011). Fast rule identification and neighborhood selection for cellular automata. *IEEE Trans. Syst. Man Cybern. B Cybern.*, 41(3):749–760. https://doi.org/10.1109/TSMCB.2010.2091271.

[5] Faizal Hafiz, Amelia Kunze, Enrico Formenti, and Davide La Torre, (2025). Identification of cellular automata on spaces of bernoulli probability measures. https://doi.org/10.48550/arXiv.2506.22867.

[6] Guy Castagnoli, James L. Massey, Philipp A. Schoeller, and Niklaus von Seemann. (1991). On repeated-root cyclic codes. *IEEE Trans. Inform. Theory*, 37(2):337–342.

[7] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, M. Pawan Kumar, Emilien Dupont, Francisco J. R. Ruiz, Jordan S. Ellenberg, Pengming Wang, Omar Fawzi, Pushmeet Kohli, and Alhussein Fawzi. (2024). Mathematical discoveries from program search with large language models. *Nature*, 625:468–475. https://doi.org/10.1038/s41586-023-06924-6.

[8] Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog, (2025). AlphaEvolve: A coding agent for scientific and algorithmic discovery. https://doi.org/10.48550/arXiv.2506.13131.

[9] Yoichi Ishibashi, Taro Yano, and Masafumi Oyamada, (2026). Effective harness engineering for algorithm discovery with coding agents. https://doi.org/10.48550/arXiv.2605.15221.

[10] Golly Project, (2026). Golly: An open source, cross-platform application for exploring cellular automata. https://golly.sourceforge.io/. Online documentation; accessed 17 October 2026.

[11] The Sage Developers, (2026). Elementary cellular automata: SageMath reference manual. https://doc-gitlab.sagemath.org/html/en/reference/dynamics/sage/dynamics/cellular_automata/elementary.html. Online documentation; accessed 17 October 2026.

[12] Wolfram Research, (2026). Cellularautomaton: Wolfram language documentation. https://reference.wolframcloud.com/language/ref/CellularAutomaton.html. Online documentation; accessed 17 October 2026.